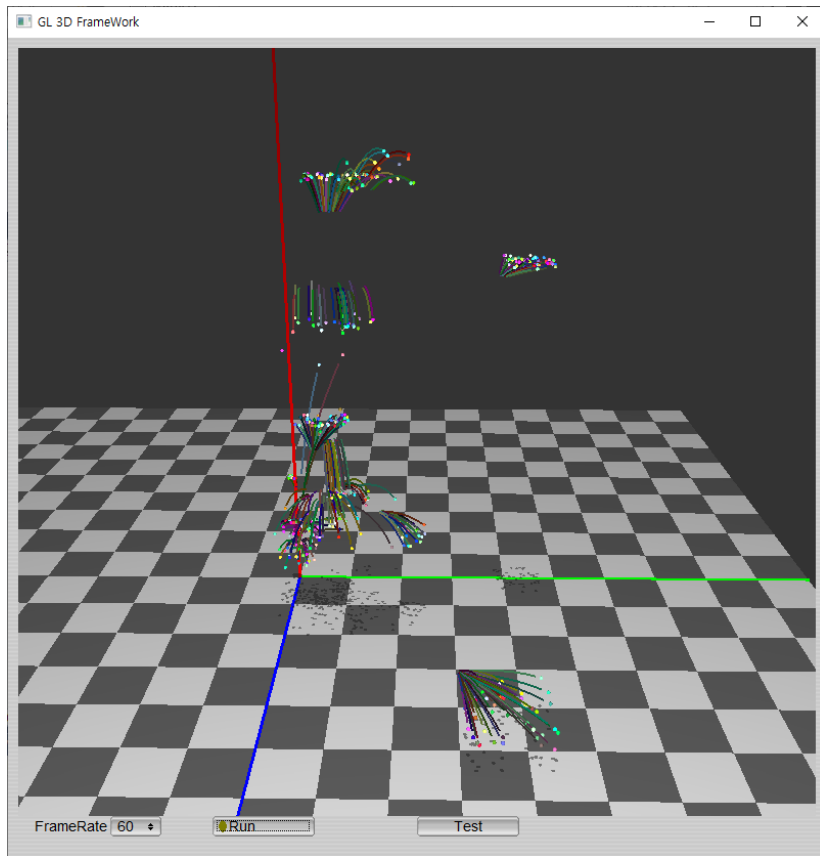


불꽃놀이 시뮬레이션

1st Assignment : Mini project



Mini Project

- ▶ 제출일은 학생들의 진도 상황을 보면서 결정
- ▶ 부분점수가 있으니, 미 완성이더라도 되는데 까지 해서 제출, 단 반드시 컴파일, 실행 에러는 없어야

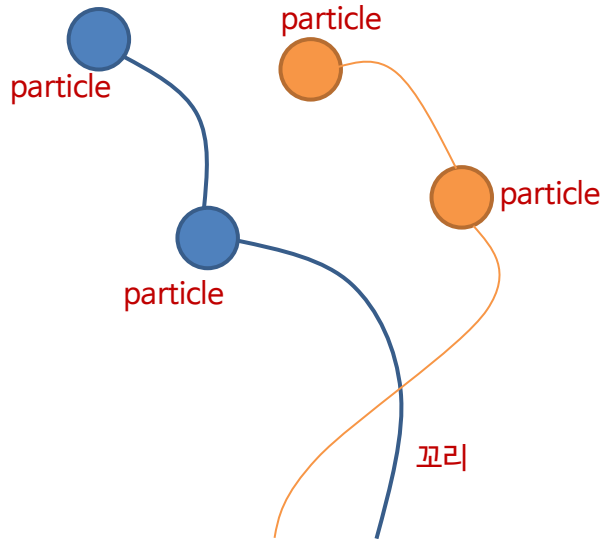


WARNING!!!

- ▶ 이 숙제는 C++의 기본개념을 잘 알아야지만 구현 할 수 있습니다.
 - ▶ 클래스 디자인
 - ▶ 기본 STL (Standard Template Library)
 - ▶ 만일 모르면 검색을 통해 익힐 것
 - ▶ object Instancing
 - ▶ 포인터
- ▶ C++에 익숙하지 않은 학생들은 빠른 시간 내에 스스로 공부하도록
 - ▶ 이 과목은 학생들이 C++를 **안다고 가정하고 진행합니다.**
 - ▶ 복잡한 C++가 필요한 것은 아닙니다. 가장 기본적인 클래스 생성(파일 단위), 컨테이너, 간단한 STL
- ▶ 클래스 디자인은 나름대로 새로 디자인 해도 됨 (슬라이드에 있는 내용을 굳이 따라 하지 않아도 됨)

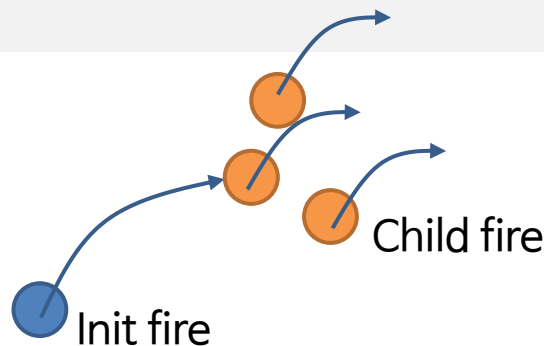
Requirement

- ▶ N개의 불꽃 생성 -> 이 N개의 불꽃이 random 시간 지난 후에 터짐 -> 다른 불꽃 생성
- ▶ 본 과제의 목적은 불꽃놀이 시뮬레이션을 제작 후, 본인이 생각하는 가장 화려한 불꽃놀이 장면을 연출 하는 것임.
 - ▶ 필요에 따라 formation 등 본인이 디자인
- ▶ 발사체가 아니므로, 발사체 느낌이 나면 절대 안됨
 - ▶ 누가 보더라도 불꽃놀이라는 생각이 들어야 함
- ▶ 각 불꽃들은 line으로 이루어진 꼬리를 보여야 함
- ▶ Padlet올라와 있는 예전 결과물 참고



Overview

- ▶ 1. Fire는 Init Fire와 Child fire로 나뉨
 - ▶ Init Fire : 발사대에서 처음 발사된 fire
 - ▶ Child Fire : 초기 Fire로부터 spawn된 fire
 - ▶ 이를 구분하기 위한 int type이 존재 함 (type = 0 (init fire) type = 1 (child fire))
- ▶ 2. Fire는 물리 법칙에 의해 움직이며 각 fire는 age가 있어서 $\text{age} < 0$ 이거나 바닥 밑으로 들어가면 사라짐
 - ▶ Fire의 초기 age는 초기에 고정 값으로 혹은 랜덤 하게 부여됨
 - ▶ Fire를 업데이트 할 때 마다 age를 duration만큼 줄어 줌
- ▶ 3. Init Fire의 age가 수명을 다하는 순간 Child Fire를 rule에 의해 생성
- ▶ 4. 생성해야 할 클래스는 총 3개 (필요에 따라 마음대로 추가/삭제 가능)
 - ▶ Class Fire : 개별 각각의 Fire
 - ▶ Class FireworkRule : Fire들의 물리적 속성을 설정하기 위한 클래스
 - ▶ Class Fireworks : 전체 Fire들을 관리 하기 위한 클래스



```
class Fire
{
public:
    Fire(int type); //type을 가진 Fire 생성 (0=Init Fire, 1=Child fire)
    ~ Fire();

    float m_size; //그리기 위한 Fire의 크기 (0.5)
    int m_type; //Fire의 타입 (set in 생성자)
    float m_age; //Fire의 현재 age
    cyclone::Particle * m_particle;
    FireworksRule * m_rule; //어떤 룰에 적용을 받나
    cyclone::Vector3 m_color; //Fire의 색상

    std::deque<cyclone::Vector3> m_history; //위치 history저장
    bool update(float duration); //전과는 다르게 bool를 반환함
    void draw(int shadow);
    void drawHistory();
    void setRule(FireworksRule * r); //특정 룰을 설정
    void putHistory(); //현재 위치를 m_history에 저장
};
```

```
class Fireworks
{
public:
    Fireworks();
    ~Fireworks();
    void update(float duration); //Fire의 생성/소멸/업데이트
    void create(); //init Fire 생성
    void create(Fire * parent); //Child Fire생성
    void draw(int shadow); //그리기
    FireworksRule m_rule[3]; //룰
    std::vector<Fire *> fireworks; //Fire 저장
};
```

본인 구현에 따라 달라질 수 있음!

```
#pragma once

#include <deque>
#include "particle.h"

class FireworksRule
{
public:
    FireworksRule() {};
    ~FireworksRule() {};

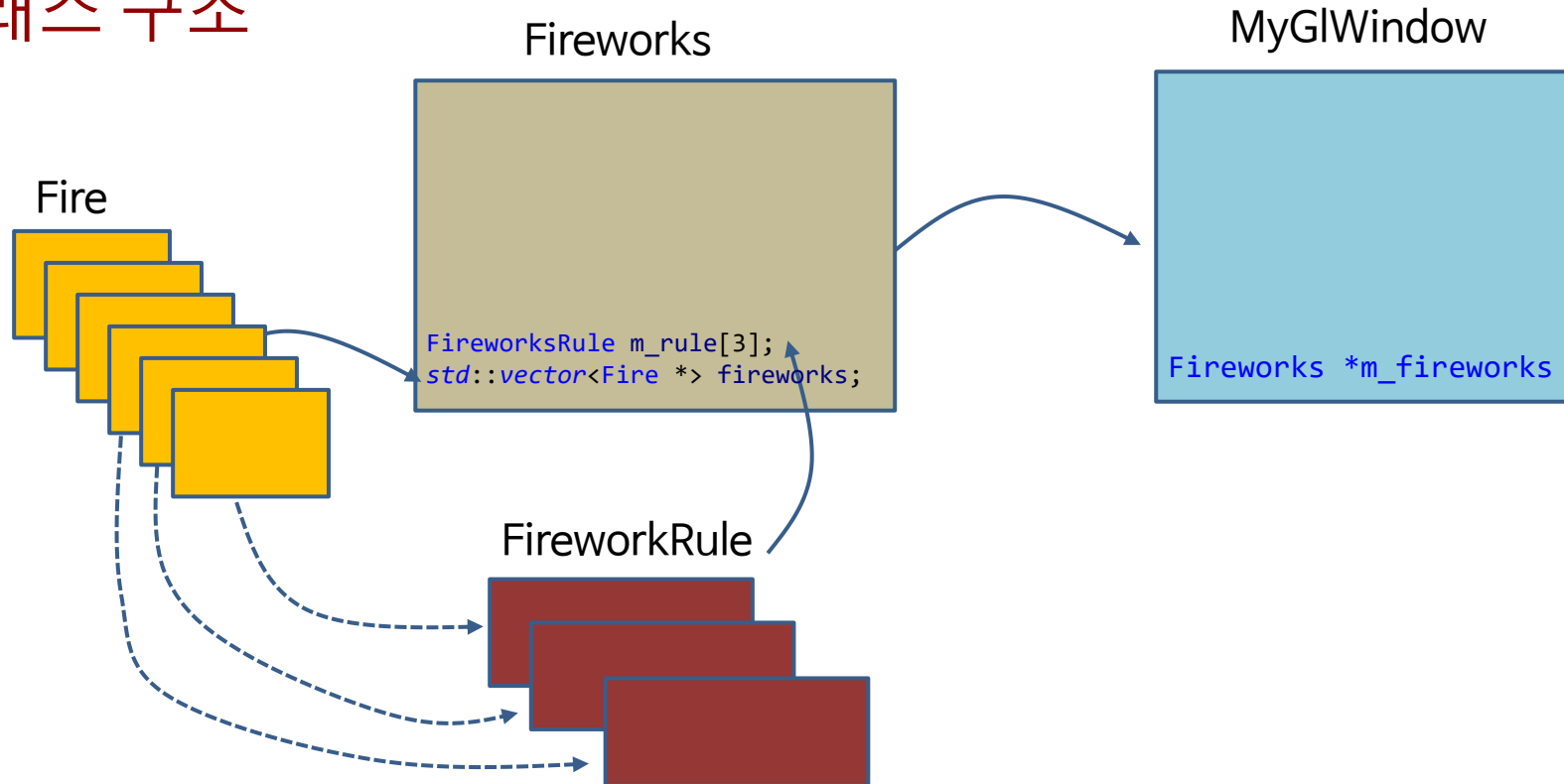
    unsigned type; //Fire의 타입
    cyclone::real minAge; //Fire의 최소 age
    Cyclone::real maxAge; //Fire의 최대 age
    cyclone::Vector3 minVelocity; //Fire의 최소 속도
    cyclone::Vector3 maxVelocity; //Fire의 최대 속도
    cyclone::real damping; //Fire의 damping

    unsigned payloadCount; //만일 ChildFire를 생성한다면 몇 개나 생성하나?

    void setParameters(unsigned type, cyclone::real minAge, cyclone::real maxAge,
        const cyclone::Vector3 &minVelocity, const cyclone::Vector3 &maxVelocity,
        cyclone::real damping, int count);
};
```

본인 구현에 따라 달라질 수 있음!

클래스 구조



STL vector / STL deque

- ▶ 본 과제를 위해서는 STL (Standard Template) 에서 제공하는 Vector와 deque를 이용함
 - ▶ 헤더파일
 - ▶ `#include <vector>` : 일종의 일반적인 컨테이너(우리가 아는 벡터형이 아님!)
 - ▶ `#include <deque>` : 자료구조 중 queue를 위한 컨테이너
 - ▶ 주요 사용법
 - ▶ 생성하는 방법
 - `std::vector<Fire *> fireworks;`
 - `std::deque <cyclone::Vector3> m_history;`
 - ▶ i번째의 element를 구하는 방법
 - `Fire * m = fireworks[i];`
 - `cyclone::vector3 p = m_history[i];`
 - ▶ 데이터 삽입
 - `fireworks.push_back(m); // Fire * m = new Fire();`
 - `m_history.push_back(pos); //cyclone::vector3 pos(0,0,0);`
 - ▶ 현재 크기를 구하기
 - `Unsigned int size = fireworks.size();`
 - `Unsigned int size = m_history.size();`

STL vector / STL deque

- ▶ Loop안 에서 stl 벡터의 특정 element를 조건에 따라 삭제 하는 방법

```
std::vector<Fire * >::iterator iter; //반복연산자 (std::vector의 개별 개체를 가르치는 포인터)

for (iter = fireworks.begin(); iter != fireworks.end(); ) { //fireworks안의 모든 fire에 대해
    Fire * m = *iter; //하나 하나씩 꺼내
    if (조건) { //만일 조건에 맞다면..
        iter = fireworks.erase(iter); //fireworks에서 삭제 함
    }
    else {
        ++iter;
    }
}
```

AGE가 0보다 작거나 땅에 닿으면 삭제 해야 하기 때문에 필요

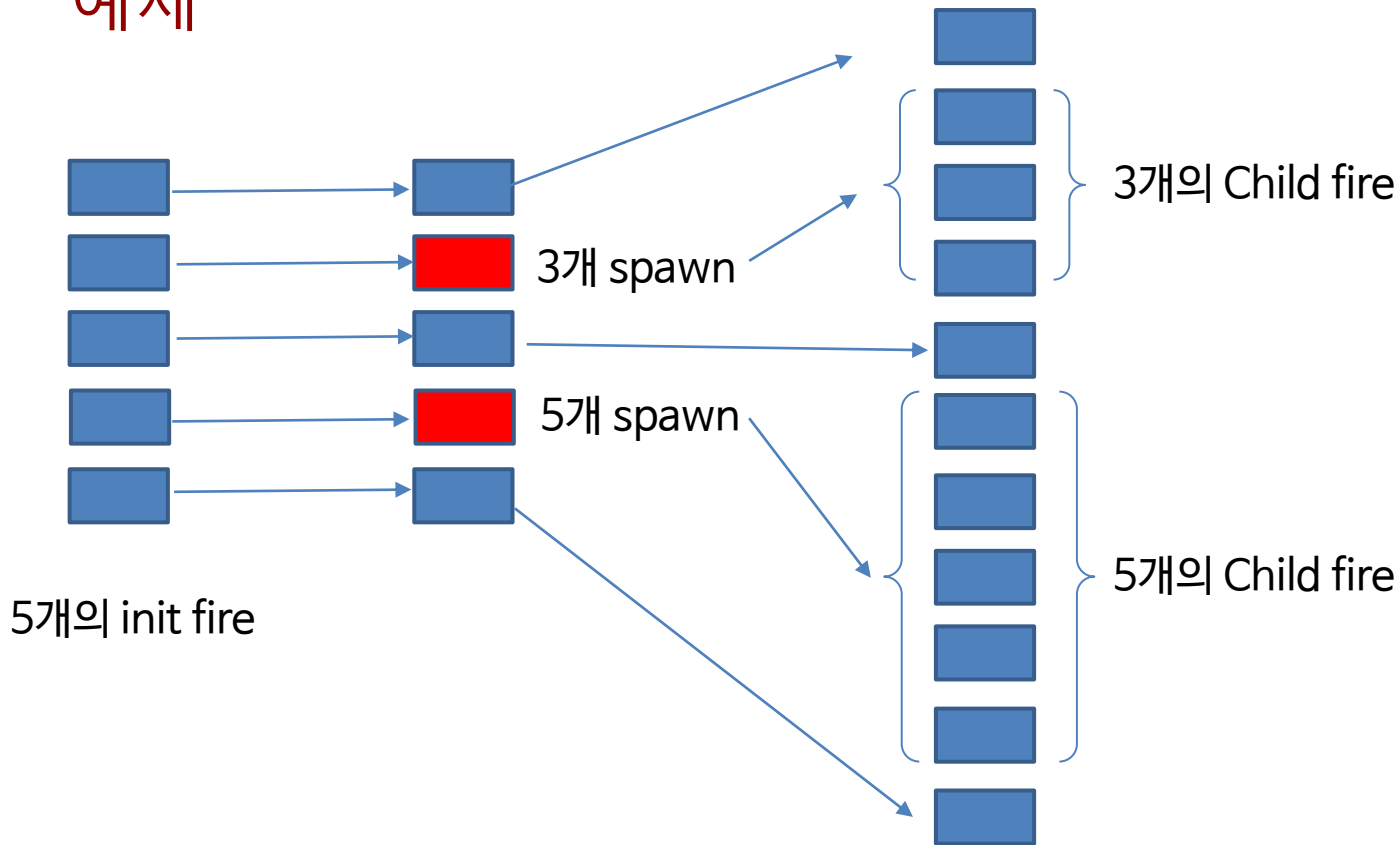
STL vector / STL deque

- ▶ Queue의 크기를 정해 놓고 삽입 하는 방법

```
if (m_history.size() < 30) {  
    m_history.push_back(p);  
}  
else {  
    m_history.pop_front();  
    m_history.push_back(p);  
}
```

Queue의 크기를 30으로 정해 놓고 30
이 넘으면 앞의 데이터를 삭제 함으로서
30을 유지

예제



랜덤 float, 랜덤 벡터 생성 하는 방법

- ▶ Cyclone에서는 랜덤 넘버 및 랜덤 벡터를 생성하기 위한 클래스를 제공

```
static cyclone::Random crandom;
```

```
//0.5 = min, 3.0 = max 사이의 랜덤 값
```

```
m_age = crandom.randomReal(0.5f, 3.0f);
```

```
//(0,0,0) = min vector, (1,1,1) = max vector 사이의 랜덤 벡터
```

```
m_color = crandom.randomVector(cyclone::Vector3(0, 0, 0), cyclone::Vector3(1, 1, 1));
```

Fireworks 클래스의 update()

- ▶ Fireworks의 update(float duration)함수에서는
 - ▶ duration을 이용하여 fireworks 벡터 안의 모든 Fire들을 업데이트 함
 - ▶ for loop를 이용
 - ▶ 만일 각 fire의 update(float duration)의 반환 값이
 - ▶ True 라면
 - 타입을 살펴서, init fire라면 rule에 따라 child Fire를 생성 후 삭제
 - 타입을 살펴서, Child fire 라면 그냥 삭제

Fireworks 클래스의 Create()함수 들

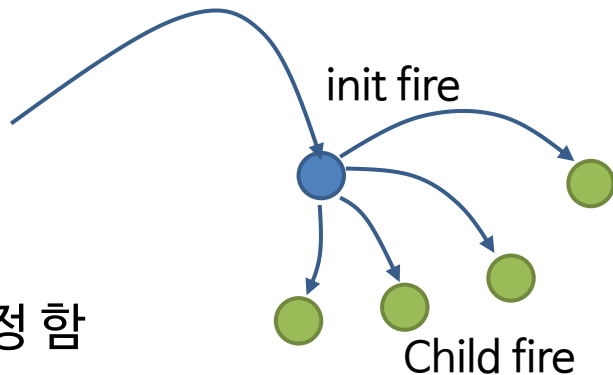
- ▶ Fireworks::Create(Fire *parent)
 - ▶ parent를 부모로 하는 child fire 생성
- ▶ Fireworks::Create()
 - ▶ Init fire 생성

Firework의 클래스의 Create() : init fire 생성

- ▶ n개의 Init Fire 생성 (type = 0)
- ▶ Fire의 물리적 속성 설정
- ▶ std 벡터에 init fire 삽입

Fireworks 클래스의 Create(Fire * parent) : Child Fire 생성 함수

- ▶ Child Fire는 자기의 부모에 해당하는 init fire (parent) 에 대한 정보를 가지고 있어야 함.
 - ▶ Child Fire의 초기 위치 = 자기 부모인 init Fire의 사망 시 위치
 - ▶ Child Fire 초기 속도 = 자기 부모인 init Fire의 사망 시 속도 + 랜덤 속도
- ▶ 연동된 FireworksRule에 따라 childfire 생성
 - ▶ Age
 - ▶ 랜덤속도
 - ▶ Mass
 - ▶ Damping
 - ▶ 몇 개나 생성 할 것인지, 그리고 타입
 - ▶ 는 모두 FireworksRule에 의해 결정 됨
- ▶ Child Fire의 초기 가속도는 중력 가속도로 설정 함

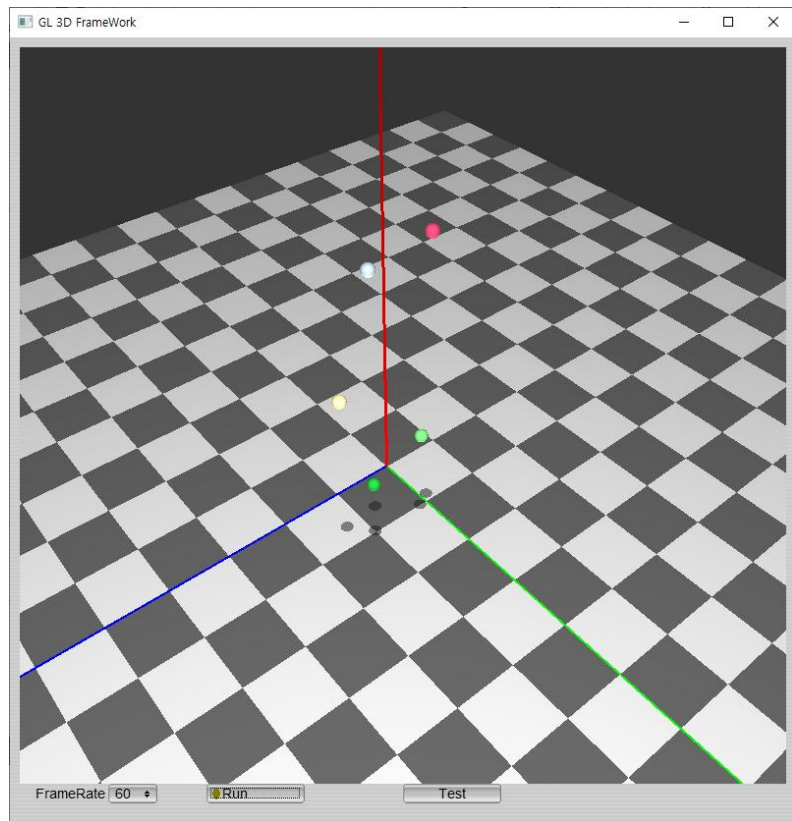


0 단계

STL 벡터 연습

아래를 구현하세요 (1) : Fire 클래스는 기존의 Mover클래스를 이름 변경

- ▶ test 버튼을 누를때 마다 5개의 fire를 생성 후에 `std::vector<Fire *> fireworks`에 저장
 - ▶ `std::vector<Fire *> fireworks`는 `MyGLWindow`의 멤버 변수
- ▶ Fire클래스의 생성자에서 `m_particle`의 물리 속성 설정
 - ▶ 파티클의 초기 위치
 - ▶ x좌표 : $-5 \leq x \leq 5$ 사이의 랜덤
 - ▶ y 좌표: 10
 - ▶ z좌표 : $-5 \leq x \leq 5$ 사이의 랜덤
 - ▶ 파티클의 초기 속도
 - ▶ `vector(0,0,0)`와 `vector(10,30,10)`사이의 랜덤 벡터
 - ▶ 파티클의 `mass = 1.0`
 - ▶ 파티클의 `damping = 0.99`
 - ▶ 파티클의 `size = 1.0`
 - ▶ 파티클의 색상
 - ▶ `vector(0,0,0)`와 `vector(1,1,1)`사이의 랜덤 벡터
 - ▶ 파티클의 초기 가속도 : GRAVITY
- ▶ `MyGLWindow`의 `update()`및 `draw()`함수에서는 FOR LOOP를 이용해 `std::vector<Fire *> fireworks`의 각 Fire들의 `update` 및 `draw()` 호출
 - ▶ `fireworks`안의 전체 Fire 숫자는 `size()`함수를 이용해서 얻는다.



1 단계

m_image추가 및 FireWorks클래스 추가

Fireworks클래스 생성

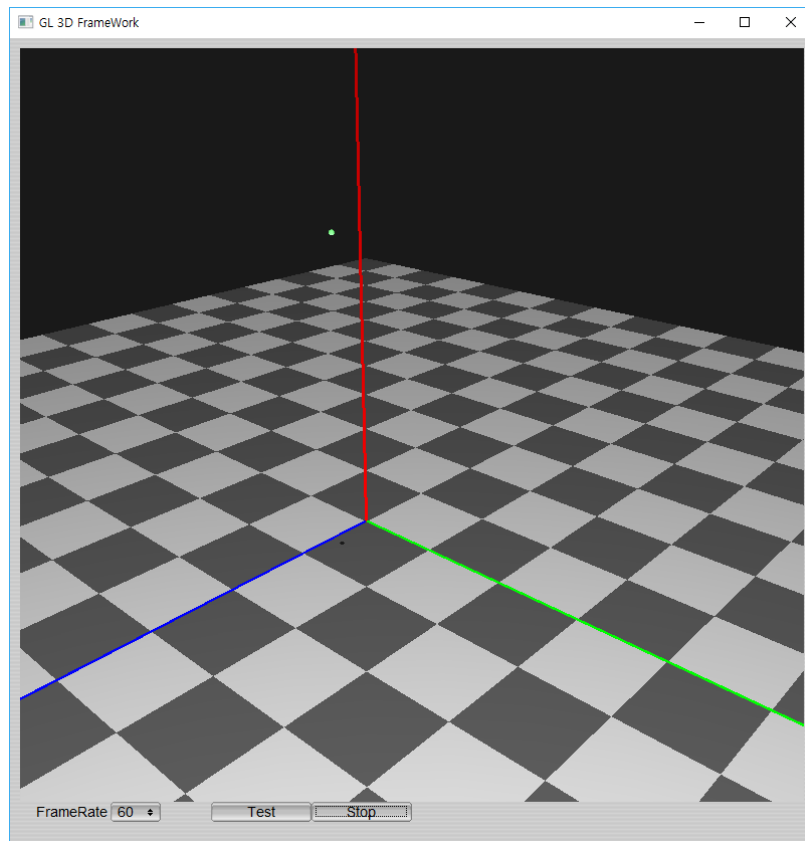
- ▶ FireWorks 클래스 디자인 (앞 슬라이드 참고)
- ▶ MyGLWindow는 FireWorks 타입을 가진 m_fireworks 멤버 변수 생성
 - ▶ MyGLWindow의 멤버 변수인 `std::vector<Fire *> fireworks`를 FireWorks클래스의 멤버 변수로 변경
- ▶ 5개의 fire생성은 FireWorks클래스의 create()함수를 통해 생성 (버튼 누를 때 마다)
- ▶ MyGLWindow의 draw()및 update()는 FireWorks클래스의 draw() 및 update()함수 호출
 - ▶ FireWorks클래스의 draw() 및 update()함수에서는 FOR LOOP를 이용해 각 Fire의 draw() 및 update()함수 호출
- ▶ Fire클래스에 m_age 추가
 - ▶ m_age는 (0.5, 3.0) 사이의 랜덤 값 부여

조건에 따른 Fire 삭제 (Fireworks::update())

- ▶ Fire클래스의 update()함수에서는 호출될 때 마다 m_age를 duration만큼 뺌
- ▶ Fireworks클래스의 update()에서는 각 Fire의 m_age < 0 혹은 y의 위치가 < 0 라면 해당 Fire를 std::vector<Fire *> fireworks로 부터 삭제 함
 - ▶ 수명을 다하거나, 만일 바닥 밑으로 가면 삭제 하는 것임

```
std::vector<Fire * >::iterator iter; //반복연산자

for (iter = fireworks.begin(); iter != fireworks.end(); ) { //fireworks안의 모든 fire에 대해
    Fire * m = *iter; //하나 하나씩 꺼내
    if (조건) { //만일 조건에 맞다면..
        iter = fireworks.erase(iter); //fireworks에서 삭제 함
    }
    else {
        ++iter;
    }
}
```



2 단계

Child Fire를 생성

Fireworks 클래스의 수정

- ▶ Update(float duration)
 - ▶ 초기 Init Fire가 생명을 다 하는 순간 Create(Fire *parent) 호출 (init Fire는 삭제)
 - ▶ 주의) Init Fire에 대한 정보를 삭제 전에 저장 해야 함
- ▶ Create(Fire * parent)
 - ▶ Init Fire인 입력 parent를 이용하여 5개의 Child Fire 생성
 - ▶ 5개 Child Fire의 위치는 init Fire의 위치 이용
 - ▶ 5개 Child Fire의 속도는 init Fire의 속도 + 랜덤 벡터
 - (cyclone::Vector3(-5, 10, -5)와 cyclone::Vector3(5, 15, 5) 사이의 랜덤 벡터)
 - ▶ 가속도는 중력가속도
 - ▶ 질량은 1.0
 - ▶ Damping은 0.9
 - ▶ Age은 랜덤 (0.5, 12.0) 사이

주의 할 점

- ▶ std::vector에 저장되어 있는 init Fire를 중간에 조건에 따라 삭제 한 후 child fire를 생성 시 주의

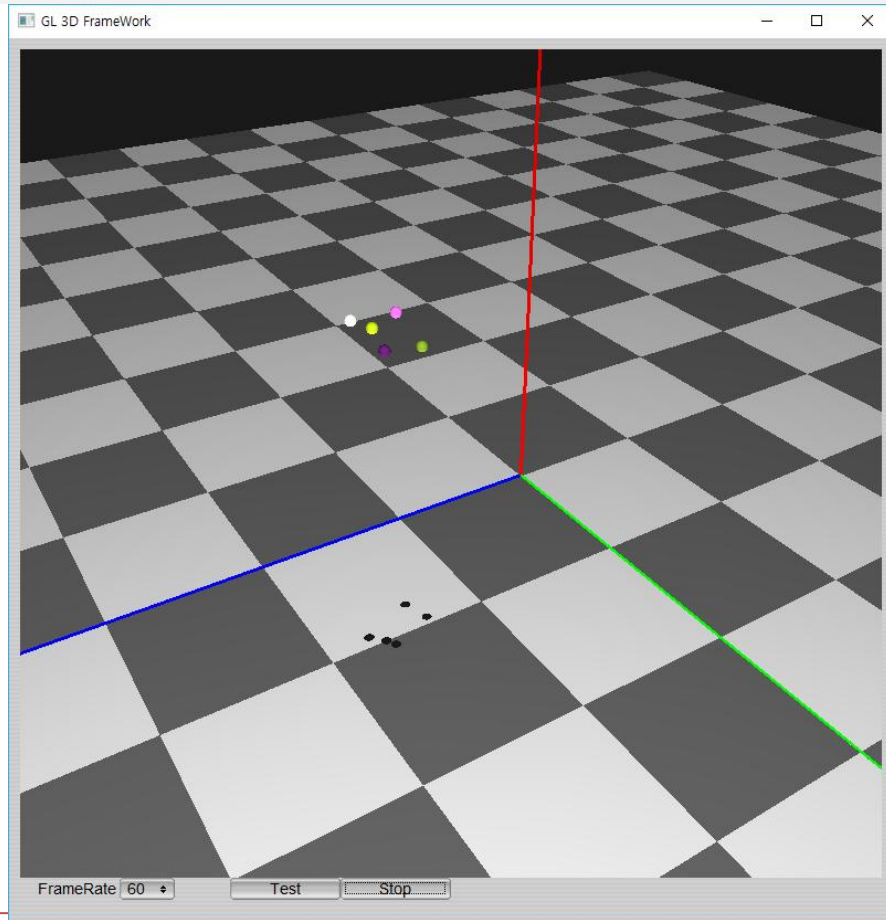
```

std::vector<Fire * >::iterator iter;
std::vector<Fire * > remove;
for (iter = fireworks.begin(); iter != fireworks.end(); ) {
    Fire * m = *iter;
    if (m->update(duration) ) {
        if (m->m_type == 0)
            remove.push_back(m);
        iter = fireworks.erase(iter);
    }
    else {
        ++iter;
    }
}

```

Init fire에 대한 정보가
있어야지만 child fire를
생성 할 수 있으므로 (예.
마지막 위치 정보) 삭제
전에 보관 해야 함

//remove에 있는 정보를 이용하여 child fire생성



3 단계

FireworksRule 이용 및 Trace 구현

Fireworks 클래스 수정

- ▶ Fireworks rule에 대한 멤버 변수 추가
 - ▶ `FireworksRule m_rule[3];`
 - ▶ 3개의 룰은 임의로 설정 (`setParameter()` 이용)
- ▶ 각 Child Fire에 3~5개의 룰 중 하나를 랜덤하게 설정하며, 물리적 속성을 rule에 따라 설정
 - ▶ 예)

```
m_rule[0].setParameters(1, 0.5f, 5.4f, cyclone::Vector3(-5, 10, -5), cyclone::Vector3(5, 15, 5), 0.9, 20);  
m_rule[1].setParameters(1, 0.5f, 10.4f, cyclone::Vector3(-15, 5, -5), cyclone::Vector3(15, 6, 15), 0.2, 40);  
m_rule[2].setParameters(1, 0.5f, 12.4f, cyclone::Vector3(-5, 0, -5), cyclone::Vector3(5, 5, 5), 0.7, 25);
```

Init Fire 생성 시

- ▶ 3개의 rule 중에 하나를 랜덤하게 선택하여 설정

```
Fire * m = new Fire(0);  
m->setRule(&m_rule[rand() % 5]);
```

- ▶ Init fire가 사라질 때, Child Fire를 생성해야 하며, 이 Child Fire의 물리적 속성은 지정된 rule에 따름
 - ▶ Age
 - ▶ 랜덤 속도
 - ▶ Damp
 - ▶ ...

Fire 클래스 수정

- ▶ 업데이트 시 위치 정보를 m_history에 저장

- ▶ 히스토리 그리기

```
void Fire::drawHistory()
{

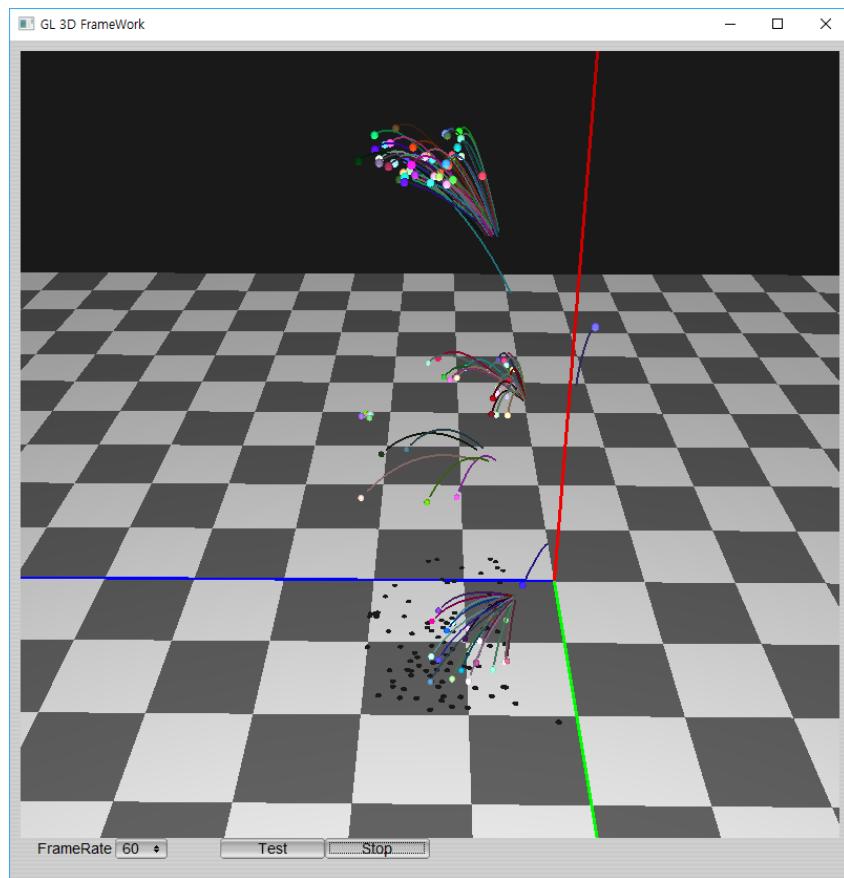
    glLineWidth(2.0f);
    glPushMatrix();

    glBegin(GL_LINE_STRIP);
    for (unsigned int i = 0; i < m_history.size(); i+=2) {
        cyclone::Vector3 p 설정
        glVertex3f(pos.x, pos.y, pos.z);
    }
    glEnd();
    glPopMatrix();
    glLineWidth(1.0f);

}
```


썸 구성

- ▶ N개 이상의 init Fire를 동시에 생성 하여 불꽃 썸 생성



제출 시

- ▶ CMakeLists.txt 및 모든 cpp, h파일을 하나의 zip파일로 압축
 - ▶ 압축파일이름 : 학번_이름.zip
 - ▶ 예) 134234_성만규.zip
 - ▶ 폴더 중에 Build, Debug, Release, .vs 반드시 삭제 후에 압축 (임시 폴더)

<https://padlet.com/mksung89/final>